

Synthese avant/apres - ms-analyse

#1. Contexte

`ms-analyse` est le microservice charge de:

- consommer des fenetres de trafic normalisees depuis RabbitMQ
- calculer des indicateurs metier
- publier les sorties utiles vers les autres consommateurs
- persister les donnees dashboard dans sa base dediee
- exposer une API REST de consultation

Les livrables ci-dessous excluent volontairement Sonar, comme demande.

#2. Rapports generes

- `03\_rapport\_tests/rapport\_tests.txt`
- `03\_rapport\_tests/rapport\_tests.xml`
- `03\_rapport\_tests/coverage.xml`
- `04\_analyse\_avant/snyk\_avant.txt`
- `04\_analyse\_avant/flake8\_avant.txt`
- `05\_analyse\_apres/snyk\_apres.txt`
- `05\_analyse\_apres/flake8\_apres.txt`

#3. Resultats de tests

Execution `pytest`:

- 16 tests executes
- 16 tests en succes
- 0 echec

Coverage:

- taux observe: 98.19%
- fichier produit: `03\_rapport\_tests/coverage.xml`

Conclusion tests:

- la couverture est largement superieure au seuil de 80%
- le service reste fonctionnel apres les corrections de securite et de dependances

#4. Analyse flake8

Etat avant remediation dediee:

- 50 ecarts releves
- principales categories:
 - E501: 34
 - W391: 13
 - E402: 2
 - E302: 1

Interpretation:

- la dette `flake8` est essentiellement cosmetique et de maintenabilite
- il n'y a pas ici de preuve d'une passe complete de nettoyage de style

Etat apres lot courant:

- le rapport detaille a ete capture dans `05\_analyse\_apres/flake8\_apres.txt`
- ce lot n'avait pas pour objectif principal de rendre `flake8` totalement vert

#5. Analyse Snyk

Etat avant:

- vulnerabilites observees sur des dependances Python signalees dans le baseline:
 - `fastapi`
 - `starlette`
 - `anyio`
 - `h11`
 - `zipp`

Correctifs appliques:

- verrouillage de versions sures et compatibles dans `requirements.txt`
- rebuild du conteneur `ms-analyse`
- verification applicative apres changement

Etat apres:

- la validation definitive Snyk necessite un environnement authentifie avec `SNYK\_TOKEN`
- un rapport textuel d'etat a ete prepare dans `05\_analyse\_apres/snyk\_apres.txt`

#6. Verifications runtime

Verifications effectuees:

- rebuild Docker du service
- demarrage de `analysis-db`, `rabbitmq` et `ms-analyse`
- verification du healthcheck HTTP

Resultat:

- `ms-analyse` démarre correctement
- `GET /health` repond `{"status":"ok"}`

#7. Conclusion generale

Avant correction:

- dependances Python vulnérables signalées par Snyk
- pipeline CI/CD a durcir
- service Docker lance initialement en root
- plusieurs alertes de fiabilité et de maintenabilité dans les tests

Après correction:

- dependances verrouillees et compatibles
- CI/CD renforcee
- utilisateur non-root dans le conteneur
- tests passes
- coverage > 80%
- service Docker verifie operationnel

Point restant:

- une passe dediee de remediation `flake8` reste recommandee pour obtenir un rendu qualite plus propre
- le scan Snyk final doit etre confirme en environnement authentifie